

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN

SORTING 2

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 25 Mei 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai SORTING 2.

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 29 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 ShellSort.....	3
2.2 QuickSort	5
2.3 MergeSort.....	8
2.4 BubbleSort (GUI).....	11
BAB III KESIMPULAN	20
3.1 Kesimpulan	20
DAFTAR PUSTAKA	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam era digital saat ini, efisiensi pengolahan data menjadi salah satu aspek krusial dalam pengembangan perangkat lunak. Salah satu operasi fundamental dalam struktur data adalah pengurutan atau *sorting*, yaitu proses mengatur elemen-elemen data dalam urutan tertentu, baik ascending (menaik) maupun descending (menurun) [1]. Kemampuan untuk mengimplementasikan algoritma *sorting* secara tepat tidak hanya memengaruhi performa aplikasi, tetapi juga menjadi dasar pemahaman untuk materi lanjutan seperti pencarian data (*searching*), basis data, dan analisis kompleksitas algoritma.

Pemahaman terhadap prinsip kerja masing-masing algoritma *sorting* tidak cukup hanya melalui teori, melainkan memerlukan praktik langsung melalui implementasi kode. Melalui praktikum ini, mahasiswa diajak untuk mengimplementasikan algoritma Quick Sort, Merge Sort, dan Bubble Sort dalam bahasa Java, baik dalam bentuk program konsol maupun antarmuka grafis (GUI). Visualisasi langkah-langkah *sorting* melalui GUI, khususnya pada Bubble Sort, membantu mahasiswa memvisualisasikan proses perbandingan dan pertukaran elemen secara interaktif, sehingga konsep abstrak algoritma dapat dipahami secara lebih konkret [2].

1.2 Tujuan Praktikum

1. Memahami konsep dasar, prinsip kerja, dan karakteristik dari berbagai algoritma pengurutan (*sorting*), meliputi Shell Sort, Quick Sort, Merge Sort, dan Bubble Sort GUI.
2. Mengimplementasikan algoritma-algoritma *sorting* tersebut ke dalam kode program menggunakan bahasa pemrograman Java, baik dalam bentuk program berbasis konsol maupun antarmuka grafis (GUI).
3. Mengembangkan kemampuan dalam membuat visualisasi langkah demi langkah proses *sorting* dengan menggunakan pustaka Java Swing

sehingga laur perbandingan dan pertukaran data dapat diamati secara interaktif.

4. Melatih kemampuan problem solving debugging, serta pengujian program secara sistematis, terstruktur, dan sesuai dengan penulisan kode yang baik.

1.3 Manfaat Praktikum

1. Meningkatkan pemahaman konseptual dan praktis mengenai algoritma sorting, serta mengasah kemampuan logika pemrograman, analisis efesiensi algoritma, dan pengembangan aplikasi berbasis GUI.
2. Menjadi fondasi yang kuat untuk mengikuti mata kuliah lanjutan.
3. Memberika pengalaman langsung menulis, mengoptimalkan dan memvisualisasikan algoritma, sekaligus membiasakan mahasiswa dengan praktik pemrograman yang terstruktur, mudah dibaca, dan sesuai dengan konvensi penamaan yang konsisten.

BAB II

PEMBAHASAN

2.1 ShellSort_2511532015

```
1 package pekan8_2511532015;
2
3 public class ShellSort_2511532015 {
4     public static void ShellSort_2511532015(int[] A) {
5         int n_2015 = A.length;
6         int gap = n_2015 / 2;
7         while (gap > 0) {
8             for (int i_2015 = gap; i_2015 < n_2015; i_2015++) {
9                 int temp = A[i_2015];
10                int j_2015 = i_2015;
11                while (j_2015 >= gap && A[j_2015 - gap] > temp) {
12                    A[j_2015] = A[j_2015 - gap];
13                    j_2015 = j_2015 - gap;
14                }
15                A[j_2015] = temp;
16            }
17            gap = gap / 2;
18        }
19    }
20
21    public static void main(String[] args) {
22        int[] data_2015 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23        System.out.print("Sebelum : ");
24        printArray(data_2015);
25        ShellSort_2511532015(data_2015);
26
27        System.out.print("Sesudah (Shell Short) : ");
28        printArray(data_2015);
29    }
30    public static void printArray(int[] arr) {
31        for (int i_2015 : arr) System.out.print(i_2015 + " ");
32        System.out.println();
33    }
34 }
```

Gambar 2.1.1: Kode Program Shell Sort

Program ini mengimplementasikan algoritma pengurutan Shell Sort menggunakan bahasa Java. Shell Sort merupakan pengembangan dari Insertion Sort yang memungkinkan pertukaran elemen yang berjarak jauh (gap), sehingga dapat mengurangi jumlah pergeseran data dan mempercepat proses pengurutan, terutama pada data yang belum terurut sebagian.

2.1.1 Method utama ShellSort_2511532015 (int[] A)

- Inisialisasi parameter dan gap awal: variable `n_2015` menyimpan panjang array, sedangkan `gap` diinisialisasikan dengan nilai setengah dari panjang array ($n_{2015}/2$). Gap ini menentukan jarak antar elemen yang akan dibandingkan pada iterasi pertama.
- Loop utama pengurang gap: perulangan `while` memastikan algoritma terus berjalan selama jarak perbandingan masih positif. Setiap kali satu siklus perbandingan selesai, nilai `gap`

akan dibagi dua hingga mencapai 1, yang mengubah algoritma menjadi Insertion Sort di tahap akhir.

- Perulangan for dimulai dari indeks gap hingga akhir array. Elemen pada indeks i_2015 disimpan dalam variable temp sebagai nilai acuan yang akan disimpan ke posisi yang tepat.
- Kemudian perulangan while memeriksa dua kondisi: apakah indeks j_2015 masih memungkinkan pergeseran ke kiri dan apakah elemen di posisi j_2015-gap lebih besar dari temp. Jika kondisi keduanya terpenuhi, elemen tersebut digeser ke posisi j dan indeks j dikurang gap.
- Setelah loop pergeseran berakhir, nilai temp ditempatkan pada indeks j yang telah selesai, menyelesaikan satu langkah penyisipan untuk elemen tersebut.

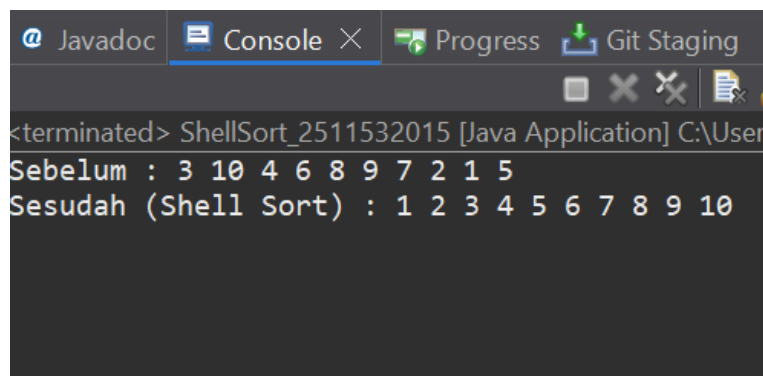
2.1.2 Method printArray

Berfungsi untuk menampilkan isi Array ke konsol menggunakan for loop untuk iterasi yang lebih ringkas dan aman dari kesalahan indeks. Setiap elemen dicetak diikuti spasi agar output lebih rapi dan System.out.println () dipanggil diakhir untuk membuat baris baru yang memisahkan output antar pemanggil method.

2.1.3 Main method

- Menginisialisasikan array berisi 10 bilangan bulat
- Memanggil method ShellSort lalu mencetak kembali keadaan array setelah proses selesai.

Ketika dijalankan program akan menampilkan output:



```
<terminated> ShellSort_2511532015 [Java Application] C:\User
Sebelum : 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort) : 1 2 3 4 5 6 7 8 9 10
```

Gambar 2.1.2: Hasil output kode program ShellSort_2511532015

2.2 QuickSort_2511532015

```
package pekan8_2511532015;

public class QuickSort_2511532015 {
    static void swap_2015 (int[] arr, int i, int j)
    {
        int temp_2015 = arr[i];
        arr[i]= arr[j];
        arr[j]= temp_2015;
    }
    //method tambahkan untuk mengatur pivot menggunakan median
    of three
    static void medianOfThree_2015(int[] arr, int low, int high)
    {
        int mid_2015 = low + (high - low) / 2;

        //urutkan elemen low, mid, dan high
        if (arr[low]> arr[mid_2015]) {
            swap_2015 (arr, low, mid_2015);
        }
        if (arr[low]> arr[high]) {
            swap_2015 (arr, low, high);
        }
        if (arr[mid_2015]> arr[high]) {
            swap_2015(arr, mid_2015, high);
        }
        swap_2015(arr, mid_2015, high);
    }
    static int partition_2015(int[] arr, int low, int high) {
        //panggil fungsi mediaOfThree sebelum menentukan pivot
        medianOfThree_2015(arr, low, high);

        int pivot = arr[high]; //sekarang arr[high] sudah berisi
        nilai median
        int i = (low - 1);

        for (int j = low; j <= high - 1; j++) {
            //jika elemen saat ini lebih kecil dari atau sama
            dengan pivot
            if (arr[j] < pivot){
                //inorement indeks elemen yang lebih kecil
                i++;
                swap_2015 (arr, i, j);
            }
        }
        swap_2015 (arr, i+1, high);
        return (i + 1);
    }
    static void quickSort_2015 (int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition_2015 (arr, low, high);
            quickSort_2015(arr, low, pi - 1);
            quickSort_2015(arr, pi + 1, high);
        }
    }
}
```

```

public static void printArr(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    int[] arr = {10, 7, 8, 9, 1, 5};
    int N = arr.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr(arr);

    quickSort_2015(arr, 0, N - 1);

    System.out.print("Data terurut quickSort: ");
    printArr(arr);
}
}

```

Algoritma Quick Sort ini bekerja dengan memilih sebuah elemen pivot, kemudian mempartisi array menjadi dua sub array berdasarkan perbandingan nilai elemen terhadap pivot. Pada implementasi praktikum, media of three diterapkan untuk pemilihan pivot.

2.2.1 Method swap_2015: berfungsi untuk menukar posisi dua elemen dalam array

- Menerima tiga parameter: array arr serta dua indeks i dan j yang akan dipertukarkan
- Nilai arr[i] disimpan sementara dalam variable temp untuk mencegah kehilangan data saat penyimpanan
- Nilai arr[j] kemudian dipindahkan ke arr[i], dan nilai temp ditempatkan ke arr[j] sehingga pertukaran posisi selesai.

2.2.2 Method medianOfThree: untuk memilih nilai pivot

- Menghitung indeks tengah (mid_2015) dari rentang low hingga high menggunakan rumus $low + (high - low)/2$
- Mengurutkan nilai pada posisi low, mid, dan high
- Setelah terurut nilai median ditukarkan ke posisi high

2.2.3 Method partition_2015: berfungsi untuk membagi array menjadi dua bagian berdasarkan pivot

- Memanggil method `medianOfThree` untuk menyiapkan pivot yang optimal di posisi `high`
- Nilai pivot diambil dari `arr[high]`, dan variable `I` diinisialisasikan sebagai penanda batas kiri elemen yang lebih kecil dari pivot ($i = \text{low} - 1$)
- Loop for dengan variable `j` menelusuri array dari `low` hingga `high-1`. Pada setiap iterasi, jika `arr[j] < pivot` maka `I` di increment dan elemen `arr[i]` ditukar dengan `arr[j]`, sehingga mengelompokkan semua elemen yang lebih kecil dari pivot ke sisi kiri.
- Setelah loop selesai, pivot ditukar dengan elemen pada posisi `i+1` sehingga pivot berada tepat di posisi akhirnya yang sudah terurut.
- Method mengembalikan indeks pivot (`i+1`) sebagai titik pemisah untuk proses rekursif selanjutnya.

2.2.4 Method `quickSort_2015`: merupakan implementasi rekursif utama dari algoritma Quick Sort

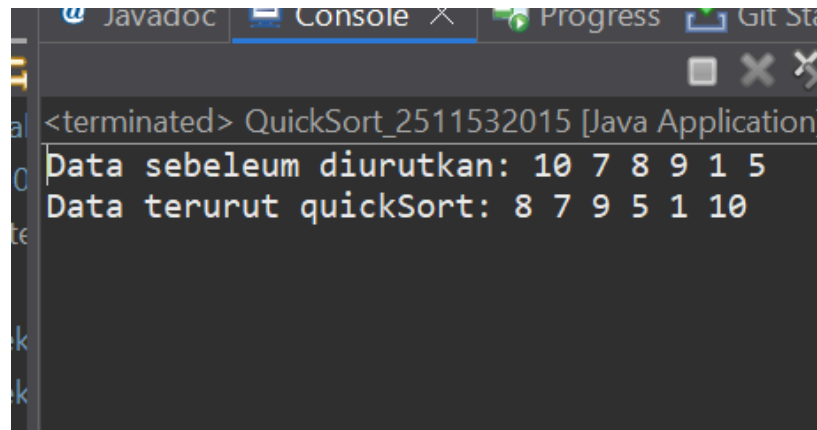
- If ($\text{lw} < \text{high}$) jika kondisi terpenuhi berarti sub array hanya memiliki satu elemen atau kosong sehingga tidak perlu diurutkan lagi
- Method `partition` dipanggil untuk memperoleh indeks pivot yang telah berada di akhirnya
- Rekursi pertama: `quickSort_2015(arr, low, pi - 1)` mengurutkan sub-array di sebelah kiri pivot.
- Rekursi kedua: `quickSort_2015(arr, pi + 1, high)` mengurutkan sub-array di sebelah kanan pivot.

2.2.5 Method `printArr`: untuk menampilkan isi array

- Menggunakan perulangan for konvensional untuk mengakses setiap elemen array berdasarkan indeks
- Setiap elemen dicetak diikuti spasi

2.2.6 Main method

- Menginisialisasikan array `arr` berisi enam bilangan bulat acak {10, 7, 8, 9, 1, 5} sebagai data uji.
- Memanggil method `quickSort_2015` untuk memulai proses pengurutan pada seluruh rentang array, sehingga ketika dijalankan akan menghasilkan output:



Gambar 2.2.1: Hasil output program QuickSort_2511532015

2.3 MergeSort_2511532015

```
package pekan8_2511532015;

public class MergeSort_2511532015 {

    void merge_2015(int arr[], int l_2015, int m_2015, int
        r_2015) {

        // Find sizes of two subarrays to be merged
        int n1_2015 = m_2015 - l_2015 + 1;
        int n2_2015 = r_2015 - m_2015;

        /* Create temp arrays */
        int L_2015[] = new int[n1_2015];
        int R_2015[] = new int[n2_2015];

        /* Copy data to temp arrays */
        for (int i_2015 = 0; i_2015 < n1_2015; ++i_2015)
            L_2015[i_2015] = arr[l_2015 + i_2015];

        for (int j_2015 = 0; j_2015 < n2_2015; ++j_2015)
            R_2015[j_2015] = arr[m_2015 + 1 + j_2015];

        int i_2015 = 0, j_2015 = 0;

        // Initial index of merged subarray array
        int k_2015 = l_2015;

        while (i_2015 < n1_2015 && j_2015 < n2_2015) {
            if (L_2015[i_2015] <= R_2015[j_2015]) {
```

```

        arr[k_2015] = L_2015[i_2015];
        i_2015++;
    } else {
        arr[k_2015] = R_2015[j_2015];
        j_2015++;
    }
    k_2015++;
}

/* Copy remaining elements of L[] if any */
while (i_2015 < n1_2015) {
    arr[k_2015] = L_2015[i_2015];
    i_2015++;
    k_2015++;
}

/* Copy remaining elements of R[] if any */
while (j_2015 < n2_2015) {
    arr[k_2015] = R_2015[j_2015];
    j_2015++;
    k_2015++;
}
}

void sort_2015(int arr[], int l_2015, int r_2015) {
    if (l_2015 < r_2015) {

        // Find the middle point
        int m_2015 = (l_2015 + r_2015) / 2;

        // Sort first and second halves
        sort_2015(arr, l_2015, m_2015);
        sort_2015(arr, m_2015 + 1, r_2015);

        // Merge the sorted halves
        merge_2015(arr, l_2015, m_2015, r_2015);
    }
}

/* A utility function to print array of size n */
static void printArray_2015(int arr[]) {
    int n_2015 = arr.length;

    for (int i_2015 = 0; i_2015 < n_2015; ++i_2015)
        System.out.print(arr[i_2015] + " ");

    System.out.println();
}

public static void main(String args[]) {

    int arr[] = {12, 11, 13, 5, 6, 7};

    System.out.println("Sebelum terurut");
    printArray_2015(arr);
}

```

```

MergeSort_2511532015 ob_2015 = new
MergeSort_2511532015();
ob_2015.sort_2015(arr, 0, arr.length - 1);

System.out.println("\nSesudah Terurut menggunakan merge
Sort");
printArray_2015(arr);
}
}

```

Algoritma Merge Sort bekerja dengan cara rekursif membagi array menjadi dua bagian hingga setiap sub array hanya terdiri satu elemen saja, kemudian menggabungkan kembali sub array tersebut secara terurut.

2.3.1 Method merge_2015: bertugas menggabungkan dua sub array yang telah terurut menjadi satu array

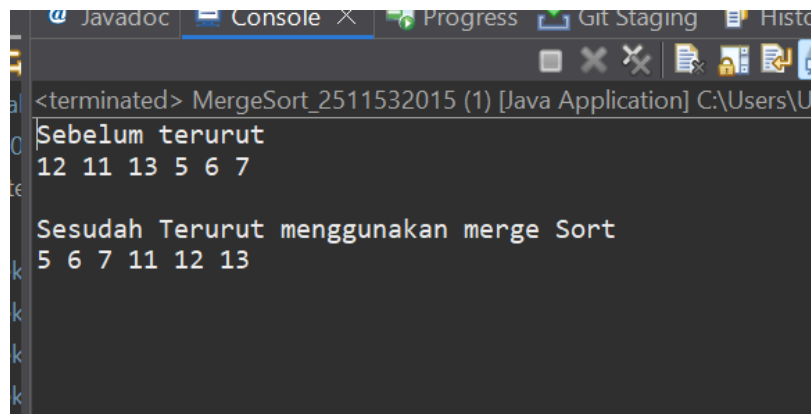
- Menerima array utama arr, indeks kiri l_2015, indeks tengah m_2015, dan indeks kanan r_2015 yang mendefinisikan batas sub array yang akan digabung
- Perhitungan ukuran sub array: variable n1_2015 menghitung jumlah elemen subarray kiri (m_2015 - l_2015 + 1), sedangkan n2_2015 menghitung elemen sub-array kanan (r_2015 - m_2015).
- Pembuatan array sementara: dua array temporer L_2015[] dan R_2015 dibuat untuk menampung Salinan data dari subarray kiri dan kanan, penggunaan array sementara ini diperlukan agar proses penggabungan tidak menimpa data asli sebelum selesai dibandingkan.
- Dua perulangan for menyalin elemen dari array utama ke array L dan R sesuai rentang indeks, memastikan data terisolasi untuk proses merging.
- Tiga variable pointer diinisialisasikan: i untuk array kiri, j untuk array kanan, dan k untuk menunjuk posisi pengisian di array utama, dimulai dari indeks l
- Perulangan while membandingkan elemen L[i] dan R[j]. elemen yang lebih kecil atau sama ditempatkan ke arr[k] kemudian pointer yang bersangkutan (i atau j) dan k di increment.

- Setelah salah satu sub array habis, dua perulangan while terpisah menyalin sisa elemen dari L atau R yang belum terproses ke array utama.

2.3.2 Method sort_2015

- Pengecekan if ($l_{2015} < r_{2015}$) memastikan proses hanya berlanjut jika sub-array memiliki lebih dari satu elemen. Jika tidak, rekursi berhenti karena array satu elemen dianggap sudah terurut.
- Indeks tengah m_{2015} dihitung dengan $(l_{2015} + r_{2015}) / 2$ untuk membagi array menjadi dua bagian yang hampir sama besar.
- Pemanggilan sort_2015 mengurutkan subarray sebelah kiri secara rekursif hingga mencapai kondisi dasar.
- Setelah kedua subarray terurut, method merge dipanggil untuk menggabungkan keduanya menjadi salah satu array terurut.

Ketika dijalankan, program akan menampilkan output:



Gambar 2.3.1: Hasil output program MergeSort_2511532015

2.4 BubbleSort_2511532015 (GUI)

```

package pekan8_2511532015;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.*;

```

```

public class BubbleSort_2511532015 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_2015;
    private JLabel[] labelArray_2015;
    private JButton stepButton_2015, resetButton_2015,
        setButton_2015;
    private JTextField inputField_2015;
    private JPanel panelArray_2015;
    private JTextArea stepArea_2015;
    private int i_2015 = 0, j_2015;
    private boolean sorting_2015 = false;
    private int stepCount_2015 = 1;

    /**
     * Create the frame.
     */
    public BubbleSort_2511532015() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2015 = new JPanel(new FlowLayout());
        inputField_2015 = new JTextField(30);
        setButton_2015 = new JButton("Set Array");
        inputPanel_2015.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
        inputPanel_2015.add(inputField_2015);
        inputPanel_2015.add(setButton_2015);

        // Panel array visual
        panelArray_2015 = new JPanel();
        panelArray_2015.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_2015 = new JPanel();
        stepButton_2015 = new JButton("Langkah Selanjutnya");
        resetButton_2015 = new JButton("Reset");
        stepButton_2015.setEnabled(false);
        controlPanel_2015.add(stepButton_2015);
        controlPanel_2015.add(resetButton_2015);

        // Area teks untuk log langkah-langkah
        stepArea_2015 = new JTextArea(8, 60);
        stepArea_2015.setEditable(false);
        stepArea_2015.setFont(new Font("Monospaced", Font.PLAIN,
14));
        JScrollPane scrollPane_2015 = new
JScrollPane(stepArea_2015);

        // Tambahkan panel ke frame
        add(inputPanel_2015, BorderLayout.NORTH);
        add(panelArray_2015, BorderLayout.CENTER);
        add(controlPanel_2015, BorderLayout.SOUTH);
        add(scrollPane_2015, BorderLayout.EAST);
    }
}

```

```

        // Event Set Array
        setButton_2015.addActionListener(e ->
        setArrayFromInput_2015());

        // Event Langkah Selanjutnya
        stepButton_2015.addActionListener(e ->
        performStep_2015());

        // Event reset
        resetButton_2015.addActionListener(e -> reset_2015());
    }

    private void setArrayFromInput_2015() {
        String text = inputField_2015.getText().trim();
        if (text.isEmpty()) return;
        String[] parts = text.split(",");
        array_2015 = new int[parts.length];
        try {
            for (int k = 0; k < parts.length; k++) {
                array_2015[k] =
                Integer.parseInt(parts[k].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
            angka "
                + "yang dipisahkan koma!", "Error",
            JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2015 = 0;
        j_2015 = 0;
        stepCount_2015 = 1;
        sorting_2015 = true;
        stepButton_2015.setEnabled(true);
        stepArea_2015.setText("");
        panelArray_2015.removeAll();
        labelArray_2015 = new JLabel[array_2015.length];
        for (int k = 0; k < array_2015.length; k++) {
            labelArray_2015[k] = new
            JLabel(String.valueOf(array_2015[k]));
            labelArray_2015[k].setFont(new Font("Arial",
            Font.BOLD, 24));
            labelArray_2015[k].setOpaque(true);
            labelArray_2015[k].setBackground(Color.WHITE);

            labelArray_2015[k].setBorder(BorderFactory.createLineBorder(
            Color.BLACK));
            labelArray_2015[k].setPreferredSize(new Dimension(50,
            50));

            labelArray_2015[k].setHorizontalAlignment(SwingConstants.CEN
            TER);
            panelArray_2015.add(labelArray_2015[k]);
        }
        panelArray_2015.revalidate();
        panelArray_2015.repaint();
    }

```

```

}

private void performStep_2015() {
    if (!sorting_2015 || i_2015 >= array_2015.length - 1) {
        sorting_2015 = false;
        stepButton_2015.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        return;
    }

    resetHighlights_2015();

    StringBuilder stepLog = new StringBuilder();
    labelArray_2015[j_2015].setBackground(Color.CYAN);
    labelArray_2015[j_2015 + 1].setBackground(Color.CYAN);

    if (array_2015[j_2015] > array_2015[j_2015 + 1]) {
        // Swap
        int temp = array_2015[j_2015];
        array_2015[j_2015] = array_2015[j_2015 + 1];
        array_2015[j_2015 + 1] = temp;
        labelArray_2015[j_2015].setBackground(Color.RED);
        labelArray_2015[j_2015 + 1].setBackground(Color.RED);
        stepLog.append("Langkah
").append(stepCount_2015).append(": Menukar elemen ke-")
            .append(j_2015).append("
").append(array_2015[j_2015 + 1]).append(" dengan ke-")
            .append(j_2015 + 1).append("
").append(array_2015[j_2015]).append("
");
    } else {
        stepLog.append("Langkah
").append(stepCount_2015).append(": Tidak ada pertukaran
antara ke-")
            .append(j_2015).append("
").append(array_2015[j_2015]).append(" dan ke-")
            .append(j_2015 + 1).append("
");
    }

    stepLog.append("Hasil:
").append(arrayToString_2015(array_2015)).append("
");
    stepArea_2015.append(stepLog.toString());
    updateLabels_2015();

    j_2015++;
    if (j_2015 >= array_2015.length - i_2015 - 1) {
        j_2015 = 0;
        i_2015++;
    }
    stepCount_2015++;

    if (i_2015 >= array_2015.length - 1) {
        sorting_2015 = false;
        stepButton_2015.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting
selesai!");
    }
}

```

```

    }

    private void updateLabels_2015() {
        for (int k = 0; k < array_2015.length; k++) {

            labelArray_2015[k].setText(String.valueOf(array_2015[k]));
        }
    }

    private void resetHighlights_2015() {
        for (JLabel label : labelArray_2015) {
            label.setBackground(Color.WHITE);
        }
    }

    private void reset_2015() {
        inputField_2015.setText("");
        panelArray_2015.removeAll();
        panelArray_2015.revalidate();
        panelArray_2015.repaint();
        stepArea_2015.setText("");
        stepButton_2015.setEnabled(false);
        sorting_2015 = false;
        i_2015 = 0;
        j_2015 = 0;
        stepCount_2015 = 1;
    }

    private String arrayToString_2015(int[] arr) {
        StringBuilder sb = new StringBuilder();
        for (int k = 0; k < arr.length; k++) {
            sb.append(arr[k]);
            if (k < arr.length - 1) sb.append(", ");
        }
        return sb.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BubbleSort_2511532015 gui = new
            BubbleSort_2511532015();
            gui.setVisible(true);
        });
    }
}

```

Program ini mengimplementasikan algoritma Bubble Sort dengan GUI menggunakan pustaka Java Swing. Tujuannya adalah untuk memvisualisasikan proses pengurutan langkah demi langkah secara interaktif, sehingga mahasiswa dapat mengamati secara langsung bagaimana perbandingan dan pertukaran elemen terjadi.

Bagian Desain Antarmuka:

Antarmuka program dibangun menggunakan pustaka Java Swing dengan mewarisi kelas JFrame dan menerapkan BorderLayout sebagai manajer tata letak utama. Jendela aplikasi dikonfigurasi dengan ukuran 750x400 piksel, judul "*Bubble Sort Langkah per Langkah*", serta posisi awal di tengah layar untuk memudahkan akses pengguna. Komponen antarmuka dibagi menjadi empat area fungsional: panel input di posisi NORTH yang berisi label petunjuk, *text field* untuk memasukkan data array, dan tombol "*Set Array*"; panel visualisasi array di posisi CENTER yang menampilkan elemen data sebagai JLabel dengan styling khusus; panel kontrol di posisi SOUTH yang memuat tombol "*Langkah Selanjutnya*" dan "*Reset*" untuk navigasi eksekusi algoritma; serta area log langkah di posisi EAST berupa JTextArea dalam JScrollPane untuk mencatat riwayat operasi secara tekstual.

Bagian Metod dan Logika Pemrograman:

2.4.1 Method konstruktor (BubbleSort_2511532015): berfungsi untuk menginisialisasikan seluruh komponen GUI dan menata letaknya sesuai zona BorderLayout, kemudian mendaftarkan event listener pada setiap tombol menggunakan lambda expression:

- `setButton_2015.addActionListener(e -> setArrayFromInput_2015())` → memicu pemrosesan input.
- `stepButton_2015.addActionListener(e -> performStep_2015())` → menjalankan satu langkah sorting.
- `resetButton_2015.addActionListener(e -> reset_2015())` → mengembalikan keadaan aplikasi.

2.4.2 Method `setArrayFromInput_2015()`: berfungsi untuk mengambil teks dari `inputField_2015`, memotong spasi ekstra dengan `trim()`, dan membagi string berdasarkan koma menggunakan `split(",")`. Setiap bagian dikonversi ke integer menggunakan `Integer.parseInt()` dalam blok try-catch untuk menangani input tidak valid.

- Menginisialisasikan `i`, `j`, dan `stepCount`

- Flag `sorting_2015` diaktifkan dan tombol “Langkah Selanjutnya” di enable
- Panel visualisasi dibersihkan, lalu dibuat ulang dengan `JLabel` baru untuk setiap elemen array
- `Revalidate()` dan `repaint()` dipanggil untuk memastikan perubahan tampilan langsung terlihat

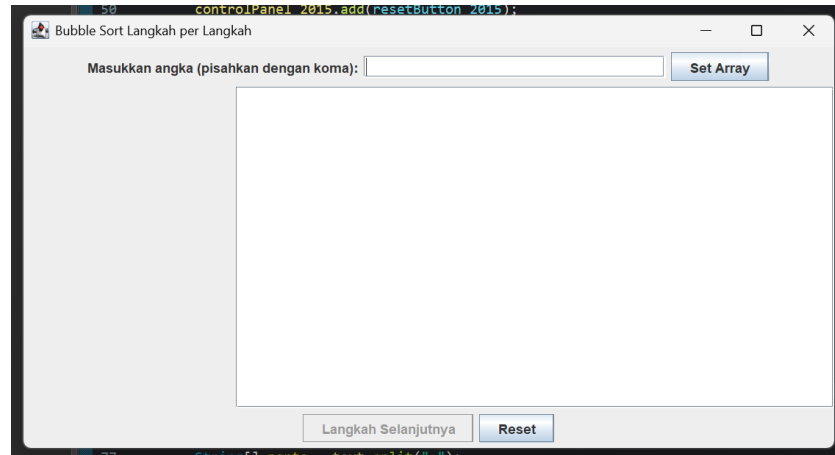
2.4.3 Method `performStep_2015()`: merupakan inti dari visualisasi algoritma Bubble Sort yang dijalankan langkah demi langkah

- Jika `sorting` bernilai false atau iterasi luar `i` sudah mencapai akhir, program menampilkan pesan “Sorting Selesai!” dan menonaktifkan tombol langkah.
- Method `resetHighligh` dipanggil untuk mengembalikan warna semua label ke putih sebelum langkah baru dimulai
- Dua elemen yang akan dibandingkan (`j` dan `j+1`) diberi latar CYAN
- `j_2015` di-increment untuk membandingkan pasangan berikutnya.
- Jika `j_2015` mencapai batas `array_2015.length - i_2015 - 1`, maka iterasi luar `i_2015` di-increment dan `j_2015` di-reset ke 0 (menandakan satu pass Bubble Sort selesai).
- `stepCount_2015` di-increment untuk penomoran log yang berurutan.

2.4.4 Main method

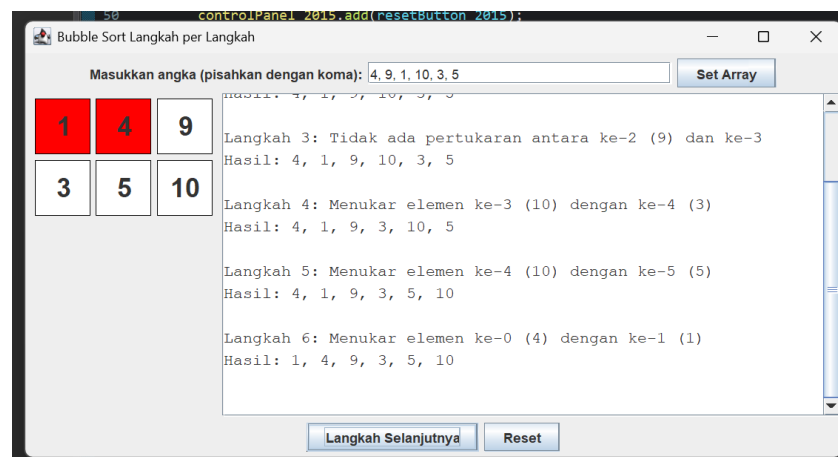
- Menggunakan `SwingUtilities.invokeLater()` untuk memastikan pembuatan dan penampilan GUI dilakukan di *Event Dispatch Thread* (EDT), sesuai praktik terbaik pemrograman Swing agar antarmuka tetap responsif.
- Membuat instance `BubbleSort_2511532015` dan memanggil `setVisible(true)` untuk menampilkan jendela aplikasi.

Ketika dijalankan program akan menampilkan output:



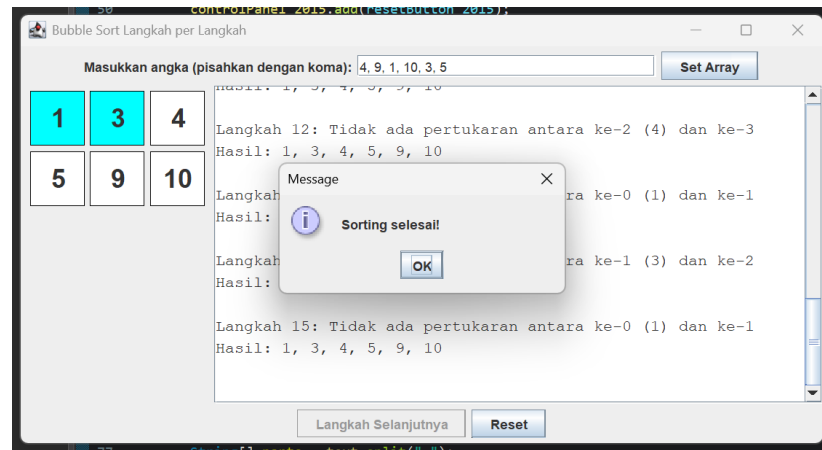
Gambar 2.4.1: Tampilan pertama program BubbleSort (GUI)

Program akan meminta user untuk memasukkan deretan angka yang dipisahkan dengan koma pada kolom input, lalu set array untuk memulai pengurutan. Jika ada karakter non angka akan muncul pesan error.



Gambar 2.4.2: Proses sorting dengan menekan tombol “Langkah Selanjutnya”

Setiap kali tombol “Langkah Selanjutnya” ditekan, program akan melakukan satu operasi Bubble Sort, proses berlanjut hingga seluruh array terurut secara manik.



Gambar 2.4.3: Ketika sorting selesai program akan menampilkan pemberitahuan “Sorting Selesai”

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Sorting Algorithms in Data Structures," GeeksforGeeks, 2026. [Online]. Available: <https://www.geeksforgeeks.org/sorting-algorithms/>. [Accessed 29 May 2026].
- [2] W3Schools, "Java Swing Tutorial – Building GUI Applications," W3Schools, 2026. [Online]. Available: https://www.w3schools.com/java/java_swing.asp. [Accessed 29 May 2026].

PENJELASAN TUGAS PEKAN 8

A. Kelas Lagu

```
1 package pekan8_2511532015;
2
3 public class Lagu_2511532015 {
4     private String judul_2015;
5     private String penyanyi_2015;
6     private int durasi_2015;
7
8     //konstruktor
9     public Lagu_2511532015 (String judul_2015, String penyanyi_2015, int durasi_2015){
10         this.judul_2015 = judul_2015;
11         this.penyanyi_2015 = penyanyi_2015;
12         this.durasi_2015 = durasi_2015;
13     }
14     //getter
15     public String getJudul_2015(){
16         return judul_2015;
17     }
18     public String getPenyanyi_2015(){
19         return penyanyi_2015;
20     }
21     public int getDurasi_2015(){
22         return durasi_2015;
23     }
24     //setter
25     public void setJudul_2015(String judul_2015){
26         this.judul_2015 = judul_2015;
27     }
28     public void setPenyanyi_2015(String penyanyi_2015){
29         this.penyanyi_2015 = penyanyi_2015;
30     }
31     public void setDurasi_2015(int durasi_2015){
32         this.durasi_2015 = durasi_2015;
33     }
34     @Override
35     public String toString() {
36         return judul_2015 + " - " + durasi_2015 + " detik";
37     }
38 }
39
```

Kelas ini berfungsi sebagai blueprint untuk mendefinisikan obyek lagu yang akan diproses dalam program, yang menyimpan atribut-atribut dasar sebuah lagu serta menyediakan metode untuk mengakses dan memanipulasi data tersebut.

B. Kelas Sorting_2511532015

```
package pekan8_2511532015;

import java.util.Scanner;

public class Sorting_2511532015 {
    private Lagu_2511532015[] dataLagu_2015 = new
    Lagu_2511532015[20];
    private int jumlahData_2015 = 0;

    //method untuk input data
    public void inputData_2015() {
        Scanner input = new Scanner (System.in);
        System.out.println("=== Masukkan Data Lagu (Minimal 7)
        ===\n");
        for (int i_2015 = 0; i_2015 < 7; i_2015++) {
            System.out.print("Masukkan judul lagu: ");
            String judul_2015 = input.nextLine();
            System.out.print("Masukkan nama penyanyi: ");
            String penyanyi_2015 = input.nextLine();
            System.out.print("Masukkan durasi (detik): ");
        }
    }
}
```

```

        int durasi_2015 = input.nextInt();
        input.nextLine();
        System.out.println();
        dataLagu_2015[i_2015] = new
Lagu_2511532015(judul_2015, penyanyi_2015, durasi_2015);
        jumlahData_2015++;
    }
}
//method pilih algoritma
public void pilihAlgoritma_2015() {
    Scanner input = new Scanner (System.in);

    System.out.println("=====");
};
    System.out.print("Pilih Algoritma (1=Shell, 2=Quick,
3=Merge): ");
    int pilihAlgoritma = input.nextInt();
}
//method algoritma (merge sort) : membagi array menjadi dua
bagian)
public void mergeSort_2015(int left, int right) {
    if (left < right) {
        int mid_2015 = left + (right - left)/2;
        mergeSort_2015(left, mid_2015);
        mergeSort_2015(mid_2015 + 1, right);
        merge_2015 (left, mid_2015, right);
    }
}
//penggabungan
private void merge_2015(int left, int mid, int right) {
    int n1_2015 = mid - left + 1;
    int n2_2015 = right - mid;

    Lagu_2511532015[] L_2015 = new Lagu_2511532015 [n1_2015];
    Lagu_2511532015[] R_2015 = new Lagu_2511532015 [n2_2015];
    for (int i_2015 = 0; i_2015 < n1_2015; i_2015++) {
        L_2015[i_2015] = dataLagu_2015 [left + i_2015];
    }
    for (int j_2015 = 0; j_2015 < n2_2015; j_2015++){
        R_2015[j_2015] = dataLagu_2015 [mid + 1 + j_2015];
    }
    int i_2015 = 0, j_2015 = 0, k_2015 = left;
    while (i_2015 < n1_2015 && j_2015 < n2_2015) {
        if
(L_2015[i_2015].getJudul_2015().compareTo(R_2015[j_2015].getJ
udul_2015()) <= 0) {
            dataLagu_2015[k_2015] = L_2015[i_2015];
            i_2015++;
        } else {
            dataLagu_2015[k_2015] = R_2015[j_2015];
            j_2015++;
        }
        k_2015++;
    }
    while (i_2015 < n1_2015) {
        dataLagu_2015[k_2015] = L_2015[i_2015];
        i_2015++;
    }
}

```

```

        k_2015++;
    }
    while (j_2015 < n2_2015) {
        dataLagu_2015[k_2015] = R_2015[j_2015];
        j_2015++;
        k_2015++;
    }
}
//method tampil data
public void tampilData_2015(String ket) {
    System.out.println(ket);
    for (int i_2015 = 0; i_2015 < jumlahData_2015; i_2015++)
    {
        if (dataLagu_2015[i_2015] != null) {
            System.out.println((i_2015 + 1) + ". " +
                dataLagu_2015[i_2015].getJudul_2015() + "
- " +
                dataLagu_2015[i_2015].getDurasi_2015() +
" detik");
        }
        System.out.println();
    }
}
public static void main(String[] args) {
    Sorting_2511532015 go_2015 = new Sorting_2511532015();
    System.out.println("=== Sorting Playlist NIM: 2511532015
===");
    go_2015.inputData_2015();
    go_2015.pilihAlgoritma_2015();
    go_2015.tampilData_2015("\nData Sebelum Sorting:");
    go_2015.mergeSort_2015(0, go_2015.jumlahData_2015 - 1);
    go_2015.tampilData_2015("\nData Setelah Merge Sort (Judul
A-Z):");
}
}

```

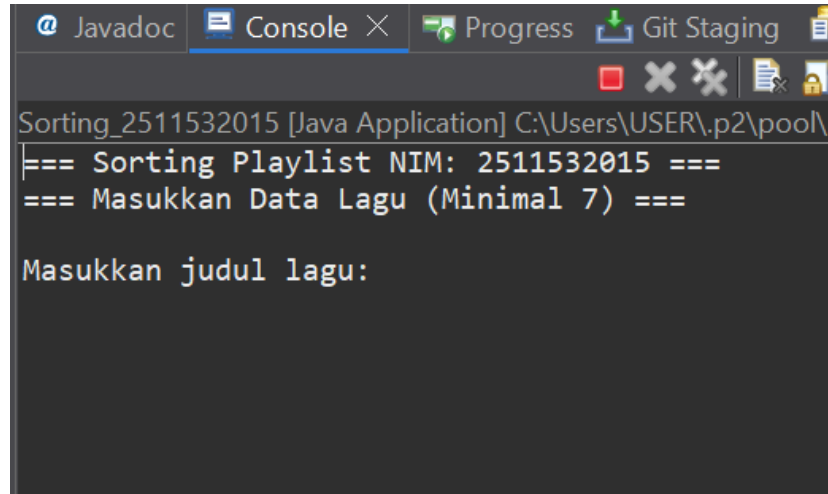
Kelas ini mengelola seluruh alur eksekusi program, mulai dari input, pemilihan algoritma, proses sorting menggunakan Merge Sort, hingga visualisasi hasil.

Method-methodnya yaitu:

- dataLagu_2015: membuat array objek Lagu_2511532015 dengan kapasitas maksimal 20 elemen untuk menyimpan data playlist
- jumlahData_2015: variable penanda untuk mencatat jumlah lagu yang tidak valid telah diinput
- inputData_2015: input user melalui Scanner untuk mengisi array dengan minimal 7 objek lagu.
- pilihAlgoritma_2015: menu untuk user memilih jenis algoritma

- mergeSort_2015: algoritma merge sort yang membagi array menjadi bagian-bagian kecil hingga tersisa satu elemen
- merge_2015: menggabungkan dua sub array yang telah terurut menjadi satu array utuh terurut berdasarkan judul
- tampilData_2015: menampilkan isi array

Ketika dijalankan program akan menampilkan output:

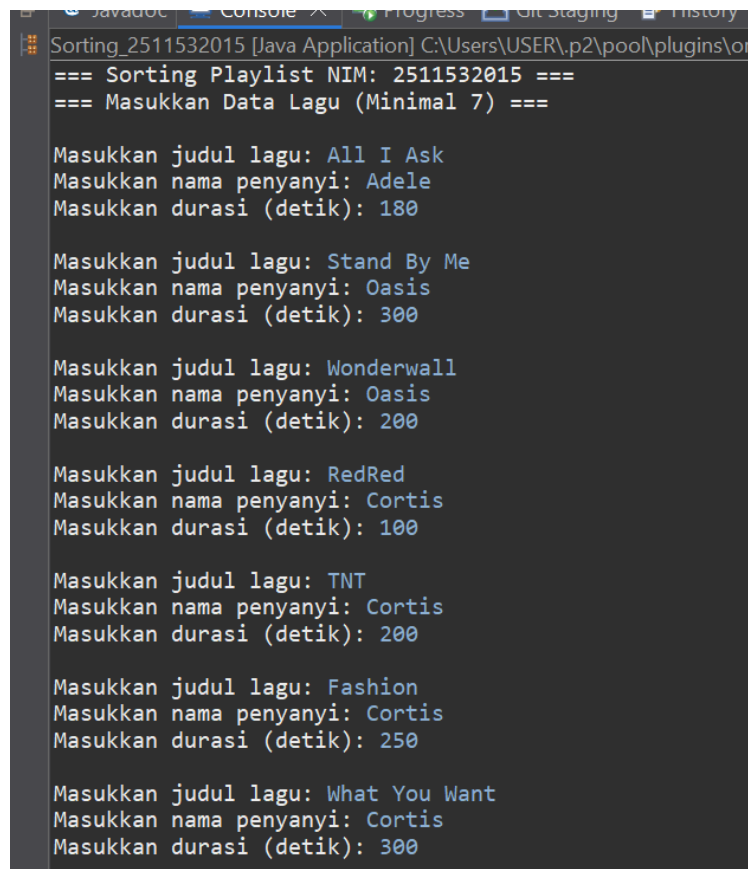


```

@ Javadoc Console X Progress Git Staging
Sorting_2511532015 [Java Application] C:\Users\USER\p2\pool\
=== Sorting Playlist NIM: 2511532015 ===
=== Masukkan Data Lagu (Minimal 7) ===

Masukkan judul lagu:
  
```

Tampilan awal ketika dijalankan



```

@ Javadoc Console X Progress Git Staging History
Sorting_2511532015 [Java Application] C:\Users\USER\p2\pool\plugins\or
=== Sorting Playlist NIM: 2511532015 ===
=== Masukkan Data Lagu (Minimal 7) ===

Masukkan judul lagu: All I Ask
Masukkan nama penyanyi: Adele
Masukkan durasi (detik): 180

Masukkan judul lagu: Stand By Me
Masukkan nama penyanyi: Oasis
Masukkan durasi (detik): 300

Masukkan judul lagu: Wonderwall
Masukkan nama penyanyi: Oasis
Masukkan durasi (detik): 200

Masukkan judul lagu: RedRed
Masukkan nama penyanyi: Cortis
Masukkan durasi (detik): 100

Masukkan judul lagu: TNT
Masukkan nama penyanyi: Cortis
Masukkan durasi (detik): 200

Masukkan judul lagu: Fashion
Masukkan nama penyanyi: Cortis
Masukkan durasi (detik): 250

Masukkan judul lagu: What You Want
Masukkan nama penyanyi: Cortis
Masukkan durasi (detik): 300
  
```

Program akan meminta user untuk memasukkan data lagu sebanyak 7 sebagai minimal untuk memulai penyortiran

```
=====
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
3

Data Sebelum Sorting:
1. All I Ask - 180 detik
2. Stand By Me - 300 detik
3. Wonderwall - 200 detik
4. RedRed - 100 detik
5. TNT - 200 detik
6. Fashion - 250 detik
7. What You Want - 300 detik

Data Setelah Merge Sort (Judul A-Z):
1. All I Ask - 180 detik
2. Fashion - 250 detik
3. RedRed - 100 detik
4. Stand By Me - 300 detik
5. TNT - 200 detik
6. What You Want - 300 detik
7. Wonderwall - 200 detik
```

Berdasarkan hasil eksekusi program yang ditampilkan pada konsol, pengguna memilih algoritma Merge Sort (opsi 3) dari menu yang tersedia untuk mengurutkan 7 data lagu yang telah diinput sebelumnya. Pada kondisi awal, data lagu ditampilkan dalam keadaan acak dengan urutan: "All I Ask", "Stand By Me", "Wonderwall", "RedRed", "TNT", "Fashion", dan "What You Want". Setelah algoritma Merge Sort dieksekusi, program berhasil mengurutkan keseluruhan data secara alfabetis (A-Z) berdasarkan atribut judul lagu, yang terbukti dari hasil output dimana "All I Ask" berada di posisi pertama, diikuti "Fashion", "RedRed", "Stand By Me", "TNT", "What You Want", dan diakhiri dengan "Wonderwall" di posisi terakhir. Hal ini menunjukkan bahwa implementasi method `mergeSort_2015()` dan `merge_2015()` telah berfungsi dengan baik dalam melakukan proses *divide-and-conquer* serta penggabungan kembali array objek Lagu_2511532015 menggunakan perbandingan string `.compareTo()` pada atribut `judul_2015`, sementara atribut `durasi_2015` tetap mempertahankan nilai aslinya dan mengikuti perpindahan posisi objek lagu yang bersangkutan, membuktikan bahwa algoritma memindahkan seluruh referensi objek secara utuh, bukan hanya atribut judulnya saja.